

**Implementación De Arboles B Y B+ De Búsqueda Externa
Informe**

Juan David Romero Morales – 20222020102

Jaider Alejandro Gonzalez Rojas - 20242020075

Jimmy Alejandro Granados Becerra - 20251020034

Juan Sebastian Gutierrez Poveda - 20251020199

Universidad Distrital Francisco Jose de Caldas

Ciencias de la Computación II – Grupo 020-84

Docente: Simar Enrique Herrera

Bogotá D.C.

16 De Agosto De 2026

Tabla de Contenido

	Pág.
1. Introducción	3
2. Objetivos.....	4
3. Explicación Código Desarrollado:	5
3.1. Nodos.....	5
<i>3.1.1. Nodo Árbol B</i>	<i>5</i>
<i>3.1.2. Nodo Árbol B+</i>	<i>5</i>
3.2. Estructura de Arboles	6
<i>3.2.1. Árbol B</i>	<i>6</i>
<i>3.2.2. Árbol B+</i>	<i>6</i>
3.3. Inserción y Eliminación.....	7
<i>3.3.1. Inserción</i>	<i>7</i>
<i>3.3.2. Eliminación</i>	<i>7</i>
4. Pruebas	8
4.1. Árbol B.....	8
4.2. Árbol B+	10

1. Introducción

LINK DEL REPOSITORIO: https://github.com/Jimmy86gb/CC2_GR5_020_84.git

En el ámbito del almacenamiento y procesamiento de datos masivos, la gestión eficiente de la memoria constituye uno de los desafíos más críticos del diseño de sistemas. A medida que los volúmenes de información superan la capacidad física de la memoria principal (RAM), los sistemas de bases de datos y de archivos deben recurrir al almacenamiento secundario (discos duros HDD o unidades de estado sólido SSD).

Sin embargo, existe una diferencia abismal de rendimiento entre ambos medios: mientras que la RAM ofrece tiempos de acceso en el orden de los nanosegundos 10^{-9} s, la lectura/escritura en disco opera en la escala de milisegundos 10^{-3} s o microsegundos. Este desfase convierte a las operaciones de Entrada/Salida (I/O) en el principal cuello de botella del rendimiento del software.

Para mitigar este impacto surge la búsqueda externa, una estrategia de diseño cuyo objetivo central no es minimizar únicamente el número de comparaciones de clave (como ocurre con la búsqueda interna en memoria), sino minimizar drásticamente el número de accesos al disco.

En este contexto, las estructuras de datos indexadas juegan un papel fundamental. Los índices basados en Árboles B y Árboles B+ fueron concebidos específicamente para este propósito. Al permitir una alta ramificación (nodos multigrado) que coincide idealmente con el tamaño de los bloques de almacenamiento del disco, estos árboles mantienen alturas sumamente bajas, posibilitando la localización, inserción y recuperación de información masiva en un número mínimo de operaciones de I/O.

2. Objetivos

2.1. Objetivo General

Diseñar, implementar y analizar cuantitativamente las estructuras de datos de búsqueda externa **Árbol B** y **Árbol B+** en el lenguaje Java bajo una arquitectura orientada al modelo Vista-Controlador (MVC) (Esto a elección nuestra).

2.2. Objetivos Específicos

1. **Implementar** en Java las clases fundacionales de representación de nodos (BTreeNode y BPlusNode dentro del paquete model) respetando la semántica de las listas de claves, listas de hijos y referencias de enlazado para el mantenimiento del orden e integridad estructural.
2. **Comparar** conceptual y operacionalmente las diferencias mecánicas entre el Árbol B tradicional (BTree) y la variante Árbol B+ (BPlusTree), enfocándose en la duplicación/promoción de claves tras los procesos de división (*split*) y la estructura del nivel hoja.
3. **Simular** el desempeño de ambas estructuras integrándolas a través de un controlador central (TreeController), analizando la respuesta del árbol ante el subdesbordamiento (*underflow*) en procesos de eliminación, los tiempos de ejecución y los recorridos de nodos.

3. Explicación Código Desarrollado

3.1. Nodos

3.1.1. Nodo Árbol B

En el proyecto, la clase BTreeNode (ubicada en el paquete co.edu.udistrital.model) representa la unidad fundamental de almacenamiento e indexación para la estructura del Árbol B. A diferencia de un árbol binario de búsqueda convencional, este nodo es multigrado y puede almacenar múltiples claves y punteros a hijos.

```
package co.edu.udistrital.model;

import java.util.ArrayList;
import java.util.List;

public class BTreeNode {

    public List<Integer> keys;

    public List<BTreeNode> children;

    public boolean isLeaf;

    public BTreeNode(boolean isLeaf) {

        this.isLeaf = isLeaf;

        this.keys = new ArrayList<>();

        this.children = new ArrayList<>();

    }

}
```

Atributos Clave

- **keys (List<Integer>):** Lista gestionada dinámicamente que almacena los valores numéricos de las claves en el nodo. Se mantiene estrictamente ordenada de forma ascendente. En un árbol de orden M, un nodo almacena hasta M-1 claves antes de requerir un proceso de división o *split*.
- **children (List<BTreeNode>):** Colección de referencias a otros objetos descendientes de tipo BTreeNode. Si el nodo es interno y posee N claves, el tamaño de esta lista será siempre N+1.

- **isLeaf (boolean):** Bandera que indica si el nodo se encuentra en la base del árbol (true) o si posee descendientes (false). Esta distinción es fundamental para detener la recursividad en búsquedas y determinar la lógica de préstamos/fusiones (*merge*) en las eliminaciones.

Manejo de Listas Ordenadas y Estructura

En la clase BTree, la lógica algorítmica hace uso directo del ordenamiento de `keys`. Los punteros en `children` subdividen los rangos de búsqueda de manera exacta: las claves del hijo `children.get(idx)` siempre serán menores a la clave en `keys.get(idx)`, enrutando eficientemente el descenso por el árbol.

3.1.2. Nodo Árbol B+

Para la estructura del Árbol B+ (clase `BPlusTree`), se diseñó la clase `BPlusNode`. Esta estructura adapta la arquitectura del nodo tradicional para optimizar drásticamente la capacidad de indexación y, sobre todo, los recorridos secuenciales (consultas por rango).

```
package co.edu.udistrital.model;

import java.util.ArrayList;

import java.util.List;

public class BPlusNode {

    public List<Integer> keys;

    public List<BPlusNode> children;

    public boolean isLeaf;

    public BPlusNode nextLeaf;

    public BPlusNode(boolean isLeaf) {

        this.isLeaf = isLeaf;
    }
}
```

```

    this.keys = new ArrayList<>();

    this.children = new ArrayList<>();

    this.nextLeaf = null;

}

}

```

Estructura Adaptada

El Árbol B+ divide semánticamente el propósito de sus nodos dependiendo de su atributo isLeaf:

- **Nodos Internos (isLeaf = false):** Exclusivamente actúan como guías de enrutamiento. Contienen claves que no representan los datos reales y referencias en children. Cuando se realiza un splitInternal, la clave promovida **sube y se elimina** del nivel actual.
- **Nodos Hoja (isLeaf = true):** Contienen los datos reales del sistema. Al dividirse (splitLeaf), la clave que asciende al padre **se mantiene duplicada** en la hoja recién creada, asegurando que el nivel base contenga el 100% de la información.

El Puntero nextLeaf y el Escaneo en Bloque

- **nextLeaf (BPlusNode):** Es la adición estructural más importante del Árbol B+. Guarda una referencia directa al nodo hoja adyacente hacia la derecha en la base del árbol.

La incorporación de nextLeaf entrelaza todas las hojas formando una **lista enlazada simple** a nivel de disco/memoria base. Gracias a esto, si se desea escanear un rango de datos o leer el archivo completo secuencialmente, el algoritmo solo requiere hacer un único descenso logarítmico desde la raíz hasta la primera hoja. Una vez en el nivel inferior, se recorre el árbol de forma puramente lineal navegando a través de los punteros nextLeaf (nodo = nodo.nextLeaf).

Esta arquitectura erradica la necesidad de realizar costosas llamadas recursivas ascendentes y descendentes (propias del Árbol B tradicional), reduciendo al mínimo absoluto las operaciones de I/O en búsquedas masivas.

3.2. Estructura de Árboles

3.2.1. Árbol B

Conexión de la raíz

BTree mantiene una única referencia pública `root` de tipo `BTreeNode`. Al contruirse el árbol (`new BTree(order)`), la raíz nace como una hoja vacía (`new BTreeNode(true)`). La raíz cambia de identidad en dos momentos:

- **Al crecer** (`insert`): cuando la raíz se llena y se divide, `insertRec` retorna un `SplitResult` con la clave promovida y el nuevo nodo hermano. Como la raíz no tiene padre que la reciba, `insert` construye manualmente un `BTreeNode` nuevo, no-hoja, con esa clave promovida y dos hijos (la vieja raíz y el nodo nuevo), y lo asigna como `this.root`. Así el árbol crece “hacia arriba”.
- **Al encogerse** (`delete`): si tras una eliminación la raíz queda sin claves pero conserva hijos, se descarta y su único hijo pasa a ser la nueva raíz (`root = root.children.get(0)`), reduciendo la altura del árbol.

Administración del orden (m) y grado mínimo

- Order es el número máximo de claves que un nodo puede llegar a tener antes de dividirse. En `insertRec`, cada vez que `node.keys.size() == order` se dispara `splitNode`.
- `MinKeys` se calcula como $(order - 1) / 2$, que es el mínimo de claves que debe conservar cualquier nodo (salvo la raíz) para respetar las propiedades del Árbol B. este valor se usa exclusivamente durante el borrado: tras eliminar un hijo, `deleteRec` verifica `node.children.get(idx).keys.size() < minKeys` y, de ser así llama a `fixUnderflow`, que intenta:
 1. Pedir prestado al hermano izquierdo (`borrowFromLeft`), si este tiene más de `minKeys`.
 2. Si no, pedir prestado al hermano derecho (`borrowFromRight`).
 3. Si ningún hermano puede prestar, fusionar (`mergeNodes`) el nodo con un hermano, absorbiendo también la clave separadora del padre.

- SplitNode calcula el punto medio dinámicamente como $(\text{keys.size()}-1)/2$, lo cual funciona tanto para órdenes pares como impares, promoviendo esa clave media al padre y repartiendo el resto entre el nodo original y el nuevo.

Lógica de búsqueda jerárquica

SearchNode es recursiva y decente:

1. En el nodo actual, avanza un índice idx mientras $\text{key} > \text{node.keys.get(idx)}$, ubicando la posición donde la clave debería estar o el subárbol donde continuar
2. Si encuentra coincidencia exacta en ese nodo, retorna true inmediatamente (a diferencia de un B+, aquí los datos –claves- pueden estar en nodos internos).
3. Si el nodo es hoja y no hubo coincidencia, retorna false (no existe una clave).
4. Si no es hoja, desciende recursivamente a $\text{node.children.get(idx)}$, que es el hijo correcto según la posición relativa de key frente a las claves del nodo (el hijo idx contiene valores entre la clave idx-1 y la clave idx).

3.2.2. Árbol B+

Separación de responsabilidades: internos vs hojas

Esta es la diferencia estructural clave frente al Árbol B, visible en BPlusNode y en la lógica de BPlusTree:

- Nodos internos (enrutadores): solo almacenan keys y children; nunca contienen datos “reales”, solo actúan como índice para decidir el camino de búsqueda. En insertRec, la rama else (nodo no-hoja) únicamente recorre node.keys para calcular idx y decide qué hijo descender –nunca inserta directamente un valor ahí-.
- Nodos hoja (datos reales): son los únicos que reciben inserciones directas de claves (node.key.add(key) ocurre solo dentro de if (node.isLeaf)). Además, cada hoja tiene el puntero nextLeaf, que enlaza las hojas entre sí en una lista enlazada horizontal –esto es

exclusivo del B+ y permite recorridos secuenciales eficientes sin necesidad de subir y bajar por el árbol.

Promoción de claves (split)

- SplitLeaf: divide la hoja por la mitad y promueve `newLeaf.keys.get(0)` al padre. Importante: esta clave se copia hacia arriba, pero también permanece en la hoja (el B+ real conserva todos los datos en las hojas; el nivel interno solo actúa como copia-índice).
- splitInternal: divide un nodo interno y promueve la clave del medio (`mid`), pero esta sí se remueve del nodo original (`internal.keys.subList(mid, ...).clear()`), coherente con que los internos solo enrutan, no "poseen" el dato.
- El enlace de hojas (`nextLeaf`) se reconecta explícitamente en `splitLeaf` (`newLeaf.nextLeaf = leaf.nextLeaf; leaf.nextLeaf = newLeaf;`) para no romper la lista enlazada al dividir.

Gestión de la raíz

Igual que en BTree, la raíz es una referencia mutable en BPlusTree:

- Nace como hoja (`new BPlusNode(true)`).
- Al crecer: si `insertRec` retorna un `SplitResult` en la raíz, `insert` crea un nuevo nodo interno con la clave promovida y los dos hijos (vieja raíz + nodo nuevo), y lo asigna como `root`.
- Al encogerse: en `delete`, hay un `while` (no un simple `if`) que colapsa la raíz repetidamente mientras `!root.isLeaf && root.keys.isEmpty() && !root.children.isEmpty()`, promoviendo el único hijo. El uso de `while` en vez de `if` sugiere que el código contempla colapsos en cadena (varios niveles vacíos seguidos), aunque conviene verificar que `deleteRec` efectivamente pueda producir ese escenario en profundidad.

3.3. Inserción y Eliminación

3.3.1. Inserción

La inserción mantiene las claves ordenadas y conserva el balance del árbol mediante un descenso recursivo. En la bibliografía este procedimiento suele denominarse `insertNonFull`; en la implementación desarrollada su equivalente es `insertRec`. La diferencia de nombre no modifica su función: localizar el hijo correcto, insertar en una hoja y propagar hacia el padre cualquier división producida por desbordamiento.

3.3.1.1 Inserción en el Árbol B

El método público `insert(int key)` inicia la operación llamando a `insertRec(root, key)`. En cada nodo se calcula un índice `idx` avanzando mientras la clave nueva sea mayor que las claves almacenadas. De esta manera, `idx` indica la posición ordenada de inserción si el nodo es hoja o el hijo por el cual debe continuar el descenso si es interno.

Cuando se alcanza una hoja, la clave se agrega con `node.keys.add(idx, key)`. Esta versión admite claves repetidas, pues no realiza una validación previa de existencia. Mientras el nodo contenga menos de `order` claves, la llamada termina sin modificar los niveles superiores. Para un árbol configurado con `order = 3`, los nodos estables tienen como máximo dos claves; la tercera provoca el desbordamiento u `overflow`.

Control del desbordamiento y división por la mediana

El desbordamiento se detecta cuando `node.keys.size() == order`. En ese instante se ejecuta `splitNode`. El punto de corte se calcula como `mid = (size - 1) / 2` y la clave ubicada en esa posición se convierte en `promotedKey`. Las claves menores permanecen en el nodo original y las mayores

pasan a un nuevo nodo hermano. Si el nodo dividido es interno, sus referencias a hijos también se reparten de modo que cada subárbol conserve el intervalo que le corresponde.

```
insert(key) → insertRec(root, key)
si el nodo es hoja: insertar en la posición idx
si hay overflow: splitNode(node) → (clave promovida, nuevo hermano)
si el nodo es interno: insertar el resultado del split en keys[idx] y children[idx +
1]
si se divide la raíz: crear una nueva raíz con los dos nodos como hijos
```

La promoción puede propagarse de forma ascendente: el padre incorpora la mediana y el nuevo hermano; si con ello también alcanza order claves, se divide a su vez. Cuando el resultado llega por encima de la raíz, insert crea una raíz interna nueva con la clave promovida y dos hijos. El árbol crece un nivel hacia arriba y todas sus hojas permanecen a la misma profundidad.

3.3.1.2 Inserción en el Árbol B+

BPlusTree aplica el mismo patrón recursivo, pero solo inserta datos en hojas. Al llegar a una hoja, comprueba `!node.keys.contains(key)`, por lo que evita duplicados, añade la clave y ordena la lista con `Collections.sort`. En nodos internos, las claves funcionan únicamente como separadores: la condición `key >= node.keys.get(idx)` envía al hijo derecho los valores iguales al separador.

La división distingue dos casos. `splitLeaf` reparte las claves desde `mid = size / 2` y copia al padre la primera clave de la hoja derecha. Esa clave no se elimina de la hoja, porque los datos reales deben permanecer en el nivel inferior. Además, enlaza `newLeaf` con la antigua hoja siguiente y después asigna `leaf.nextLeaf = newLeaf`, preservando el recorrido secuencial. En cambio, `splitInternal` promueve la clave media y sí la retira del nodo dividido, ya que los nodos internos son índices de enrutamiento. Si se divide la raíz, se crea una nueva raíz de la misma forma que en BTree.

3.3.2. Eliminación

La eliminación es más compleja porque retirar una clave puede dejar un nodo con menos claves que el mínimo permitido. Este estado se denomina subflujo o underflow. El Árbol B corrige el déficit mediante préstamos y fusiones; el Árbol B+ del proyecto utiliza un borrado más directo, centrado en las hojas y en el colapso de nodos vacíos.

3.3.2.1 Eliminación y control de subflujo en el Árbol B

`delete(int key)` comprueba primero que la raíz contenga datos y luego llama a `deleteRec`. En cada nivel se calcula `idx` de la misma manera que durante la búsqueda. Si la clave se encuentra en una hoja, se elimina directamente. Si está en un nodo interno, se reemplaza por su **sucesor**: el **menor** valor del subárbol **derecho**. Después se elimina recursivamente ese **sucesor** del nodo en el que realmente estaba almacenado.

Tras cada retorno recursivo se verifica si el hijo descendido contiene menos de $\text{minKeys} = (\text{order} - 1) / 2$. Cuando ocurre el underflow, `fixUnderflow` intenta restaurar la ocupación sin alterar el orden global. Este control ascendente permite corregir primero el nivel más profundo y propagar al padre los cambios estructurales necesarios.

Préstamo desde un hermano

La primera alternativa es redistribuir claves. `borrowFromLeft` se utiliza cuando el hermano izquierdo posee más de `minKeys` claves: la clave separadora del padre baja al inicio del hijo deficitario y la mayor clave del hermano sube al padre para reemplazarla. Si los nodos son internos, también se transfiere el último hijo del hermano izquierdo.

Si el hermano izquierdo no puede prestar, `fixUnderflow` examina el derecho. `borrowFromRight` mueve la clave separadora del padre al final del hijo deficitario y eleva al padre la menor clave del hermano derecho. En nodos internos se transfiere igualmente el primer hijo del hermano derecho. El préstamo conserva el número de nodos y evita reducir la altura.

Fusión de nodos

Cuando ninguno de los hermanos dispone de claves adicionales, `mergeNodes` fusiona dos nodos adyacentes. El nodo izquierdo incorpora primero la clave separadora extraída del padre y luego todas las claves del nodo derecho. Si son nodos internos, también incorpora sus hijos. Finalmente, el padre elimina la referencia al hermano derecho. Como el padre pierde una clave, puede quedar a su vez en underflow y requerir una corrección en el nivel superior.

```
deleteRec(node, key)
si la clave está en una hoja: eliminarla
si está en un nodo interno: sustituirla por el sucesor y borrar el sucesor
si el hijo queda bajo minKeys: prestar desde la izquierda o la derecha
si no es posible prestar: fusionar con un hermano
si la raíz queda vacía: promover su único hijo como nueva raíz
```

Al finalizar, si la raíz quedó sin claves pero conserva descendientes, su primer hijo se convierte en la nueva raíz. Este caso es la única situación en la que el Árbol B reduce su altura.

Enfoque de borrado en el Árbol B+

En BPlusTree el descenso utiliza las claves índice hasta alcanzar la hoja correspondiente y elimina allí el dato, pues los registros reales no residen en nodos internos. Si la hoja aún conserva claves, la estructura permanece. Cuando queda vacía, `deleteRec` la retira de la lista de hijos del padre y repara la cadena horizontal haciendo que la hoja anterior apunte a `child.nextLeaf`. También elimina la clave separadora asociada.

Si un nodo índice queda sin claves, el algoritmo elimina su envoltura: cuando todavía tiene un hijo, conecta ese hijo directamente con el nivel superior; si no tiene descendientes, retira el nodo y el separador correspondiente. Finalmente, delete utiliza un ciclo while para promover repetidamente el único hijo mientras la raíz interna esté vacía. Esto evita conservar niveles de índice sin información útil.

Alcance de la implementación. El borrado B+ desarrollado es más simple que el algoritmo canónico completo: elimina hojas vacías y colapsa índices, pero no calcula una ocupación mínima ni implementa préstamos o fusiones para hojas parcialmente ocupadas. Tampoco recalcula todos los separadores cuando se elimina la primera clave de una hoja que continúa existiendo. Por ello, su optimización consiste en operar directamente sobre las hojas y preservar nextLeaf, pero debe considerarse una versión didáctica; para producción sería necesario añadir redistribución, fusión y actualización sistemática de claves índice.

Costo algorítmico

Tanto la inserción como la eliminación descienden una ruta desde la raíz hasta una hoja. Con m como orden y h como altura, el recorrido visita $O(h)$ nodos; dentro de cada nodo se examinan hasta $O(m)$ claves mediante listas, para un costo $O(h \cdot m)$ en esta implementación. Como la alta ramificación mantiene h aproximadamente en $O(\log_m n)$, el objetivo práctico es reducir el número de niveles y, por tanto, los accesos a bloques de almacenamiento externo.

4. Pruebas

Para las pruebas se definió un orden 3 de árbol en el código(modificable), al igual que una lista de datos iniciales para permitir una secuencia de agregación más rápida y experimental, de igual manera se añadieron opciones para solicitar datos al usuario para inserción eliminación “manual”.

```
ARBOLES B Y B+
1. insertar datos de prueba en arbol b
2. insertar dato en arbol b
3. eliminar dato de arbol b
4. mostrar estructura del arbol b
5. insertar datos de prueba en arbol b+
6. insertar dato en arbol b+
7. eliminar dato de arbol b+
8. mostrar estructura del arbol b+
9. salir
elige una opcion:
```

4.1.Árbol B

- Uso de Opción 1 para realizar la inserción de datos preestablecidos

<pre>elige una opcion: 1 Dato a insertar: 10 Estructura del arbol b \-- [10] Dato a insertar: 20 Estructura del arbol b \-- [10, 20] Dato a insertar: 5 Estructura del arbol b \-- [10] +-- [5] \-- [20] Dato a insertar: 6 Estructura del arbol b \-- [10] +-- [5, 6] \-- [20]</pre>	<pre>Dato a insertar: 6 Estructura del arbol b \-- [10] +-- [5, 6] \-- [20] Dato a insertar: 12 Estructura del arbol b \-- [10] +-- [5, 6] \-- [12, 20] Dato a insertar: 30 Estructura del arbol b \-- [10, 20] +-- [5, 6] +-- [12] \-- [30]</pre>	<pre>Dato a insertar: 7 Estructura del arbol b \-- [10] +-- [6] +-- [5] \-- [7] \-- [20] +-- [12] \-- [30] Dato a insertar: 17 Estructura del arbol b \-- [10] +-- [6] +-- [5] \-- [7] \-- [20] +-- [12, 17] \-- [30]</pre>
---	--	--

```

Estructura del arbol b
\-- [10]
  +-- [6]
    | +-- [5]
    | \-- [7]
  \-- [20]
    +-- [12, 17]
    \-- [30]
Datos Iniciales Insertados en Arbol B

```

Se permite ver una secuencia de pasos para verificar método de inserción visto en clase.

- Uso de Opción 2 para insertar un único dato digitado por el usuario:

<pre> elige una opcion: 2 ingrese el valor a insertar en b: 67 dato insertado con exito: Estructura del arbol b \-- [10] +-- [6] +-- [5] \-- [7] \-- [20] +-- [12, 17] \-- [30, 67] </pre>	<pre> elige una opcion: 2 ingrese el valor a insertar en b: 69 dato insertado con exito: Estructura del arbol b \-- [10] +-- [6] +-- [5] \-- [7] \-- [20, 67] +-- [12, 17] +-- [30] \-- [69] </pre>
---	--

Se hizo uso del árbol ya ingresado, así mismo la inserción de dos datos para notar el caso de inserción sin necesidad de ruptura y el que sí tuvo necesidad respectivamente.

- Uso de Opción 3 para eliminar un único dato digitado por el usuario:

<pre> elige una opcion: 3 ingrese el valor a eliminar en b: 69 proceso de eliminacion finalizado. Estructura del arbol b \-- [10] +-- [6] +-- [5] \-- [7] \-- [20] +-- [12, 17] \-- [30, 67] </pre>	<pre> elige una opcion: 3 ingrese el valor a eliminar en b: 67 proceso de eliminacion finalizado. Estructura del arbol b \-- [10] +-- [6] +-- [5] \-- [7] \-- [20] +-- [12, 17] \-- [30] </pre>
--	--

Se eliminaron secuencialmente los datos anteriormente ingresados uno a uno para notar que la eliminación sin y con necesidad de merge (respectivamente) funciona y llega al árbol ingresado inicialmente.

4.2. Árbol B+

- Uso de Opción 5 para realizar la inserción de datos preestablecidos

elige una opcion: 5 Dato a insertar: 10 Estructura del arbol b+ -- [hoja] [10] Hojas enlazadas: [10] -> null Dato a insertar: 20 Estructura del arbol b+ -- [hoja] [10, 20] Hojas enlazadas: [10, 20] -> null	Dato a insertar: 5 Estructura del arbol b+ -- [indice] [10] +-- [hoja] [5] -- [hoja] [10, 20] Hojas enlazadas: [5] -> [10, 20] -> null Dato a insertar: 6 Estructura del arbol b+ -- [indice] [10] +-- [hoja] [5, 6] -- [hoja] [10, 20] Hojas enlazadas: [5, 6] -> [10, 20] -> null	Dato a insertar: 12 Estructura del arbol b+ -- [indice] [10, 12] +-- [hoja] [5, 6] +-- [hoja] [10] -- [hoja] [12, 20] Hojas enlazadas: [5, 6] -> [10] -> [12, 20] -> null Dato a insertar: 30 Estructura del arbol b+ -- [indice] [12] +-- [indice] [10] +-- [hoja] [5, 6] \-- [hoja] [10] -- [indice] [20] +-- [hoja] [12] -- [hoja] [20, 30] Hojas enlazadas: [5, 6] -> [10] -> [12] -> [20, 30] -> null
Dato a insertar: 7 Estructura del arbol b+ -- [indice] [12] +-- [indice] [6, 10] +-- [hoja] [5] +-- [hoja] [6, 7] \-- [hoja] [10] -- [indice] [20] +-- [hoja] [12] -- [hoja] [20, 30] Hojas enlazadas: [5] -> [6, 7] -> [10] -> [12] -> [20, 30] -> null	Dato a insertar: 17 Estructura del arbol b+ -- [indice] [12] +-- [indice] [6, 10] +-- [hoja] [5] +-- [hoja] [6, 7] \-- [hoja] [10] -- [indice] [20] +-- [hoja] [12, 17] -- [hoja] [20, 30] Hojas enlazadas: [5] -> [6, 7] -> [10] -> [12, 17] -> [20, 30] -> null	
Estructura del arbol b+ -- [indice] [12] +-- [indice] [6, 10] +-- [hoja] [5] +-- [hoja] [6, 7] \-- [hoja] [10] -- [indice] [20] +-- [hoja] [12, 17] -- [hoja] [20, 30] Hojas enlazadas: [5] -> [6, 7] -> [10] -> [12, 17] -> [20, 30] -> null Datos Iniciales Insertados en Arbol B+		

Se observa la correcta inserción de los datos preestablecidos, la lista de hojas enlazadas, y los nodos índices intermedios, características principales de árboles B+.

- Uso de Opción 2 para insertar un único dato digitado por el usuario:

<pre> elige una opcion: 6 ingrese el valor a insertar en b+: 67 dato insertado con exito. Estructura del arbol b+ \-- [indice] [12] +-- [indice] [6, 10] +-- [hoja] [5] +-- [hoja] [6, 7] \-- [hoja] [10] \-- [indice] [20, 30] +-- [hoja] [12, 17] +-- [hoja] [20] \-- [hoja] [30, 67] Hojas enlazadas: [5] -> [6, 7] -> [10] -> [12, 17] -> [20] -> [30, 67] -> null </pre>	<pre> elige una opcion: 6 ingrese el valor a insertar en b+: 69 dato insertado con exito. Estructura del arbol b+ \-- [indice] [12, 30] +-- [indice] [6, 10] +-- [hoja] [5] +-- [hoja] [6, 7] \-- [hoja] [10] +-- [indice] [20] +-- [hoja] [12, 17] \-- [hoja] [20] \-- [indice] [67] +-- [hoja] [30] \-- [hoja] [67, 69] Hojas enlazadas: [5] -> [6, 7] -> [10] -> [12, 17] -> [20] -> [30] -> [67, 69] -> null </pre>
---	--

Se hizo uso del árbol ya ingresado, así mismo la inserción de dos datos para notar el caso de inserción sin necesidad de ruptura y el que sí tuvo necesidad respectivamente, se nota el comportamiento de árbol b desde el penúltimo nodo hacia arriba.

- Uso de Opción 3 para eliminar un único dato digitado por el usuario:

<pre> elige una opcion: 7 ingrese el valor a eliminar en b+: 69 proceso de eliminacion finalizado. Estructura del arbol b+ \-- [indice] [12, 30] +-- [indice] [6, 10] +-- [hoja] [5] +-- [hoja] [6, 7] \-- [hoja] [10] +-- [indice] [20] +-- [hoja] [12, 17] \-- [hoja] [20] \-- [indice] [67] +-- [hoja] [30] \-- [hoja] [67] Hojas enlazadas: [5] -> [6, 7] -> [10] -> [12, 17] -> [20] -> [30] -> [67] -> null </pre>	<pre> elige una opcion: 7 ingrese el valor a eliminar en b+: 67 proceso de eliminacion finalizado. Estructura del arbol b+ \-- [indice] [12, 30] +-- [indice] [6, 10] +-- [hoja] [5] +-- [hoja] [6, 7] \-- [hoja] [10] +-- [indice] [20] +-- [hoja] [12, 17] \-- [hoja] [20] \-- [hoja] [30] Hojas enlazadas: [5] -> [6, 7] -> [10] -> [12, 17] -> [20] -> [30] -> null </pre>
---	--

Se realizaron pruebas intentando mantener las claves índices al borrar el dato de la hoja, pero no fue posible debido a que el nodo perdía su subárbol derecho y quedaba vacío, además violaba el hecho de que una clave se separe en dos ramas, debido a eso si se elimina x numero de la hoja se “recalcula” el índice como se ve.